

Board Defect Inspection AI

Task 4 — Controlled UISpec Renderer

Beginner-Friendly Enterprise Engineering Guide

Task 4 follows the deterministic validator and an architecture review. Its purpose is to create the controlled UI Runtime boundary that renders only an already-validated UISpec for the board inspection landing page.

Executive Summary

Task 4 is the proposed React + TypeScript renderer increment for the version 1.0 board UISpec contract. An explicit, developer-controlled registry maps approved component names to hand-written React components. Raw model output must never enter this render path.

Document status: implementation guide and adapted prompt only. Task 4 has not been implemented by this document work. The Task 3 report records 163 passing tests; that is historical evidence, not a new test run or acceptance of every unresolved interpretation.

The BRD motivates inspection visibility, defect reporting and traceability. This increment presents approved information; it does not inspect images, classify defects or decide whether a board passes. No Python-to-React API connection is claimed.

1. Where Task 4 Fits in the Learning Sequence

The project advances through small, separately reviewed increments.

```
Business Requirements and Discovery
↓
AI-Native Engineering / Architecture
↓
Controlled Generative UI landing-page scenario
↓
Task 1 — Project Engineering Foundation
↓
Task 2 — Machine-Checkable UISpec Contract
↓
Task 3 — Deterministic UISpec Validator
↓
Contract, Validation and Architecture Review
↓
Task 4 — Controlled UISpec Renderer
```

The schema and validator precede rendering. Review is still needed for the two schema paths, renderer failure policy and approved-input handoff. Git checkpoint readiness must be verified separately because the Task 3 report records unreadable Git objects.

2. Task 4 Objective

Build the controlled UISpec renderer for the UI Runtime. It receives only a specification that has passed the agreed validation boundary.

Correct: `render(approvedUISpec)`

Not: `render(rawAIOutput)`

Validation decides contract compliance; the renderer maps approved blocks to known implementations.

“Approved UISpec” means presentation validation, never board approval or a guarantee about all natural-language meaning.

3. The Task 4 Prompt

The following adapts the original guide's implementation brief, preserving its monospaced prompt format and ordered instructions. It is a future implementation prompt; preparing this guide does not execute it.

Proceed with Task 4: implement the Controlled UISpec Renderer.

This is the next proposed implementation increment for the Board Defect Inspection AI project, after readiness review.

IMPORTANT ARCHITECTURAL DECISION:

The renderer must receive ONLY an already-validated UISpec.

The renderer must NOT accept raw AI/model output.

VALID refers to a UISpec, never to board PASS/FAIL or approval.

Conceptually:

RAW AI OUTPUT

↓

UISpec Validator + supplied grounded context

↓

VALIDATED UISpec

↓

Controlled Renderer

↓

Approved React Components

↓

Browser UI

The renderer contract is conceptually:

`render(approvedUISpec)`

Do NOT implement `render(rawAIOutput)`.

Before making changes:

1. Inspect branch, commit, working tree and project structure.
Preserve unrelated work. Report Git failures without repair.
2. Read CLAUDE.md, README.md and the references below.

- Documents/Business Requirements Document.docx
 - docs/business-problem.md and docs/architecture.md
 - specs/spec.md and specs/model-contracts.md
 - specs/schemas/ui-spec.schema.json and its schema notes
 - specs/ui-spec.schema.json (compare the competing copy)
 - specs/TASK-3-READINESS.md
 - specs/TASK-3-IMPLEMENTATION-REPORT.md
 - services/domain/validator.py and validation_models.py
 - services/domain/text_policy.py and README.md
 - tests/validator/ and tests/README.md; evals/README.md
 - Relevant Documents/ guides and Task 2 clarifications
3. Review actual Task 1-3 files and current checks. Historical guides and the BRD's examples do not override the v1 contract. Do not reuse onboarding commits, test counts or constants.

TASK 4 GOAL

Implement a deterministic renderer for the board landing page. Translate validated blocks through an explicit component registry. Use specVersion "1.0", page "landing", contextId and blocks.

The approved component vocabulary is exactly:

- Hero
- Heading
- Text
- FeatureCard
- FeatureGrid
- CTA
- Alert
- InfoPanel
- ProgressIndicator

READINESS BEFORE IMPLEMENTATION

Confirm the authoritative schema path and its differing bounds. Task 3 loads specs/schemas/ui-spec.schema.json; normative docs also point to specs/ui-spec.schema.json. Do not silently merge them. Resolve architecture's omit-and-record wording versus this brief's reject-rendering requirement before choosing renderer behaviour. Agree how approved fixtures enter the renderer. Task 3 returns status/rejections, not an approved payload or browser trust token. Review Hero.primaryAction labelling: no action label prop exists. Do not invent a prop, URL, route or destination to fill that gap. If required decisions remain unresolved, report blockers and STOP.

IMPORTANT SECURITY / CONTROL REQUIREMENTS

The model must NOT create arbitrary executable components. Do not execute generated JavaScript, shell or other code. Do not use eval(), Function(), dangerouslySetInnerHTML, model-driven dynamic imports or arbitrary component resolution. Do not bypass the registry or silently substitute components. Use explicit approved props; never spread model props onto DOM.

RENDERER DESIGN

Use a fixed developer-authored registry with all nine entries:

```
componentRegistry = {
  Hero: HeroComponent, Heading: HeadingComponent,
  Text: TextComponent, FeatureCard: FeatureCardComponent,
  FeatureGrid: FeatureGridComponent, CTA: CTACComponent,
  Alert: AlertComponent, InfoPanel: InfoPanelComponent,
  ProgressIndicator: ProgressIndicatorComponent
}
```

Use an explicit typed registry/switch, never dynamic discovery.
An unknown component must not render. Apply the reviewed failure policy consistently; do not repair, relabel or hide violations.

VALIDATION BOUNDARY

Do not duplicate the JSON Schema or grounding rules in React.
JSON Schema -> structural validation
Task 3 validator -> grounding and deterministic policy checks
Renderer -> approved component mapping and DOM safety

Use fixed fixtures validated by the existing Python validator against their complete synthetic GroundedContext. Retain both files and verification evidence. Do not treat a TypeScript cast, brand or model-supplied approved=true flag as proof of validation. Keep arbitrary JSON entry points out of the demonstration. No API, model gateway or live data fetching belongs in this task.

REACT IMPLEMENTATION

Inspect existing tooling. Create only the minimum React/TypeScript structure in apps/ui/ needed to demonstrate and test the renderer. Use strict TypeScript and explicit props; do not introduce any. Do not build the complete board inspection application. Preserve block order and supplied text, metric values and units. Do not calculate, round, classify or infer missing information. Use semantic headings, accessible controls and text for severity.

FeatureGrid.props.items contains 2.6 FeatureCard PROPS objects. It is not a generic children array or nested block envelope. Hero and InfoPanel have no child components. InfoPanel.rows is metric-row data. ProgressIndicator uses metricId, label and value (0..100), not steps/current or metrics.shift_progress references.

Representative synthetic UISpec (validate with matching context):

```
{
  "specVersion": "1.0",
  "page": "landing",
  "contextId": "task4-demo",
  "blocks": [
    {
      "component": "Hero",
      "props": {
        "title": "Board inspection activity",
        "subtitle": "Inspection records and defect reporting."
      }
    },
    {
      "component": "Text",
      "props": {"text": "Explore recorded inspection activity."}
    },
    {
      "component": "CTA",
      "props": {
        "label": "Open defect reports",
        "action": "view-defect-reports"
      }
    }
  ]
}
```

The matching context must contain every GroundedContext field, contextId "task4-demo", page "landing", contextVersion "1.0", fixed locale/generatedAt, empty capabilities/defectCategories/metrics/alerts, and action id "view-defect-reports" with its label. These are synthetic examples, never evidence of live operations. Use the ACTUAL schema and contract for exact types and bounds.

ACTION CONTROL

The only approved action identifiers are:

- view-inspection-queue
- view-defect-reports
- view-ingest-activity
- open-documentation
- contact-support

Actions must also be available in the supplied context.

Keep action handling developer-owned. A test callback may record an identifier without navigation, network calls or state changes.

Do not invent URLs, destination routes or inspection workflows.

Any optional demonstration with inert controls must say so outside model-authored content. Resolve Hero action labels before building.

TESTING

Add small, readable deterministic renderer tests. At minimum:

1. All nine approved components resolve through the registry.
2. Hero + Heading/Text/CTA render as sibling blocks, not children.
3. FeatureGrid renders 2 and 6 card-props items in supplied order.
4. CTA renders its label and uses only approved action handling; cover all five actions and optional Hero.primaryAction.
5. Unknown names, including run_query, DispositionControl and inherited object keys, cannot resolve as registry components.
6. Model-like strings cannot execute code or inject markup.
7. Unknown props cannot become DOM attributes, event handlers, styles or executable behaviour; no arbitrary prop spreading.
8. Same approved input gives the same observable rendering.
9. Rendering and action tests leave the input unchanged.
10. A representative validated fixture traverses the complete path.
11. InfoPanel preserves values/units; ProgressIndicator represents the supplied percentage without rounding or invented figures.
12. Heading levels and Alert severity remain accessible; verify registry failure handling matches the reviewed policy.

Validate positive fixtures with Task 3. Deliberately malformed renderer-boundary probes are test-only, never a production bypass. Do not accept old refusal, root/children or generated_for shapes. Do not import 19/24/12 test counts, 30 nodes or an 8 KiB limit. Tests do not prove all implied quality verdicts are impossible.

IMPORTANT:

Do not add AI/model integration, a model gateway, FastAPI runtime or integration, authentication or domain data access.

Do not add physical camera integration, edge image processing, real-time PLC/MES handling, automated defect classification, human review/override workflows or production infrastructure. Do not build customer onboarding, registration or risk scoring. Do not add model retry/repair or fallback orchestration here. Do not redesign the contract or change the schema to suit React. If a required inconsistency remains, STOP and report it. Do not invent a new validation result API or rejection code.

BEFORE IMPLEMENTATION

First explain:

1. The minimal renderer structure and readiness findings.
 2. Where the explicit component registry will live.
 3. How verified fixtures reach the approved-input boundary.
 4. Files to create or modify, including required documentation.
 5. How determinism and non-mutation will be demonstrated.
 6. How executable content and arbitrary DOM props are prevented.
- Then implement only when the readiness decisions are resolved.

DEPENDENCY DISCIPLINE

Inspect available tooling before selecting packages. Add only React, TypeScript and necessary renderer test/build/lint tooling. Choose compatible versions at implementation time and record them. Do not copy obsolete versions from the onboarding tutorial. Keep dependency locks; ignore node_modules and build artifacts.

AFTER IMPLEMENTATION

Run the actual renderer tests and TypeScript checks.
Run the project-required prohibition/lint checks for unsafe paths.
Run the existing Task 3 suite from the repository root:
 .venv/Scripts/python.exe -m pytest -q
Confirm which Task 2 schema checks that suite actually executes:
JSON parsing, Draft 2020-12 meta-validation and instance checks
are different checks. Record actual commands and results.

Confirm:

- Renderer tests and existing regression checks pass.
- All nine components and all five actions are covered.
- No model, API, authentication or physical integration was added.
- No board quality approval, classification or override was added.
- No raw model output reaches the render entry point.
- No code execution or model-driven module loading was added.
- Schema and grounding authority remain in the existing validator.
- No live validator-to-browser transport is claimed.

Review git status, git diff --stat, git diff and new files.
If Git remains unreadable, report exact failures and inspect local files; do not claim a clean diff or repair the repository implicitly.

Do NOT commit. Do NOT push. Do NOT start Task 5.

Stop and report:

1. Files created
 2. Files modified
 3. Component registry design
 4. Renderer design and approved-input boundary
 5. Tests added
 6. Actual test results
 7. Existing regression and schema-check results
 8. Dependencies added, if any
 9. Architectural issues, limitations and current Git status
 10. Recommended next smallest task
- Wait for human review before proceeding further.

4. What Task 4 Actually Builds

4.1 React + TypeScript UI Runtime Foundation

The proposed increment establishes the UI Runtime's representation of UISpec 1.0 and a small renderer demonstration. It does not implement the whole landing-page service or the wider inspection platform.

4.2 Explicit Component Registry

The registry maps each approved name to developer-authored React code.

UISpec component name



Approved Component Registry



Developer-authored React component

The contract contains exactly these nine components:

- Hero
- Heading
- Text
- FeatureCard
- FeatureGrid
- CTA
- Alert
- InfoPanel
- ProgressIndicator

4.3 Renderer

The renderer preserves the order of top-level blocks. Only FeatureGrid nests cards, through an array of FeatureCard props. Hero has no children; InfoPanel holds rows of metric data. No arbitrary recursive component tree exists in this contract.

5. Model Contract vs Component Registry

The JSON Schema and component registry have different jobs. Neither replaces the other.

Model Contract



JSON Schema



Deterministic Validator + Grounded Context



Validated UISpec



Runtime Component Registry



React Renderer

The schema defines permitted output structure. The validator checks grounded identifiers, supplied values and implemented policy rules. The registry defines which hand-written implementations may render. TypeScript types must follow that contract, not create a competing schema.

6. Security and Control Boundaries

The renderer controls the final step from structured data to browser UI.

PROBABILISTIC AI Model – future integration



Raw UISpec

DETERMINISTIC CONTROL

UISpec Validator + supplied context



Validated UISpec

CONTROLLED UI RUNTIME

Component Registry → React Renderer → Browser

The system controls presentation capability. It does not transfer board-quality authority to a model or renderer.

6.1 Explicit Registry Instead of Dynamic Loading

Component resolution must use a fixed map of approved own entries. Model strings cannot select arbitrary modules, DOM tags or inherited object properties.

6.2 No eval / Function / Arbitrary HTML Execution

Render text as React text. Do not interpret model strings as HTML, JavaScript, CSS, SQL, shell or templates. Controlled developer-authored styles remain implementation code, not model-supplied content.

6.3 Controlled Props

Select declared properties explicitly. Do not spread an untrusted props object into a DOM element. Action identifiers map to reviewed application-owned behaviour; they are not URLs or executable handlers.

6.4 Unknown Components Are Rejected

An unregistered name must never render. The original guide requires deterministic rejection; docs/architecture.md says to omit the block and record a rejection. The implementation brief requires this difference to be resolved before coding. This guide does not silently select or rewrite that policy.

7. Deterministic Tests

Task 4 has no test results yet. The planned suite covers four areas:

- Registry integrity: exactly nine approved entries and no dynamic resolution.
- Rendering: representative v1 blocks, nested card props and action labels.
- Security: safe text, controlled DOM props and unsupported-name handling.
- Determinism: stable output, block order and input non-mutation.

7.1 What the Tests Demonstrate

- Every approved component maps to its expected implementation.
- Positive examples pass the existing validator with fixed context.
- FeatureGrid accepts card props, without inventing generic containers.
- Metric values and units are preserved; no figures are recomputed.
- Text cannot become executable HTML and props cannot add capabilities.
- Unknown components never resolve outside the fixed registry.
- Action probes use supplied test callbacks without external effects.
- The same input produces equivalent observable output without mutation.

Tests must establish each claim they make. Source scanning alone does not prove runtime safety, and keyword filtering does not prove semantic absence of all implied board-quality verdicts.

8. Regression Verification

The Task 3 implementation report records **163 passed in 13.02s** and a successful dependency check. Its suite includes selected-schema meta-validation and instance/coverage checks. These results were not rerun for this document adaptation.

Task 4 must report fresh renderer, TypeScript, lint and Python regression results. Do not copy the original guide's 19 renderer tests, 24 validator tests or 12/12 schema checks. No Task 4 completion evidence exists yet.

9. Dependencies Introduced

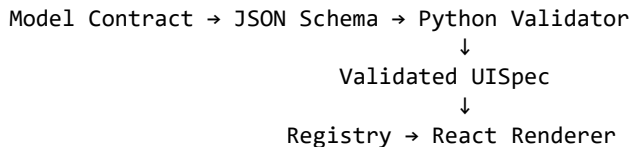
This document change adds no application dependency. Select the minimum compatible renderer toolchain only when implementation is authorised and readiness issues are resolved.

- React and react-dom for developer-authored components.
- TypeScript and matching React type definitions for strict typing.
- A small renderer test runner, such as Vitest, if selected.
- Only the test environment, build and lint support actually required.

These are proposed dependency categories, not installed packages or approved version pins. Inspect the repository first. NFR-3 calls for tests and a lint rule against unsafe execution paths; do not copy the tutorial's omission of lint tooling as a board-project exemption.

10. Task 4 Files

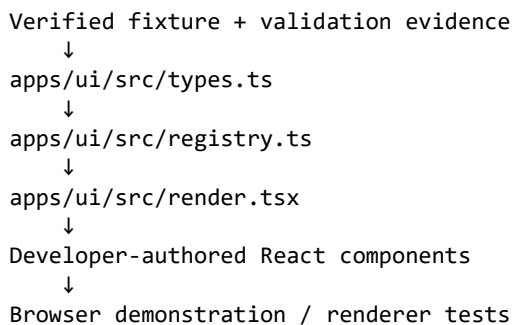
The planned UI Runtime location is **apps/ui/**, consistent with CLAUDE.md. The tutorial's ui/ and domain/ paths do not match this project.



The existing validator is services/domain/validator.py. It loads specs/schemas/ui-spec.schema.json. The alternative schema path in normative documents remains a review issue; the renderer must not quietly use a different contract.

Relationship to the Earlier Tasks

Task 2 defines structure. Task 3 validates a payload against context and returns status/rejections. Task 4 would render that same verified payload through controlled components. There is no transport, server endpoint or approved-payload wrapper implemented by this guide.



How the Files Work Together

The file structure below is a proposal, not a created application. Keep fixtures paired with synthetic context and verify them through the existing Python validator before renderer use. A brand or cast documents intent but cannot authenticate untrusted JSON.

Task 4 Test Summary

- determinism.test.tsx: repeated output, stable ordering and input non-mutation.
- security.test.tsx: safe text, controlled props and prohibited render paths.

- render.test.tsx: approved v1 component combinations and nested card props.
- registry.test.ts: exact vocabulary, fixed mapping and unknown-name handling.

Test Files

Place focused renderer tests under apps/ui/tests/. Use fixed approved fixtures and explicit expected outcomes. Do not target the original tutorial's test count.

Source / Renderer Files

- render.tsx: maps approved blocks through the registry; preserves their order.
- registry.ts: explicit mapping for all nine approved component names.
- components.tsx: hand-written components with exhaustive typed props.
- errors.ts: reviewed renderer failure representation; no invented validator codes.
- types.ts: v1 specification, props and action types; not a second validator.

Configuration Files

- README.md: boundary, fixture workflow, limitations and actual commands.
- package.json and lockfile: selected minimal dependencies and scripts.
- tsconfig.json: strict TypeScript configuration.
- vitest.config.ts, if Vitest is selected; minimal required lint configuration.

Task 4 File Structure

```
apps/ui/
├── package.json / lockfile           proposed, not created
├── tsconfig.json
├── vitest.config.ts                 if selected
├── README.md
├── src/
│   ├── types.ts
│   ├── errors.ts
│   ├── components.tsx
│   ├── registry.ts
│   └── render.tsx
└── tests/
    ├── fixtures/                   UISpec + matching context
    ├── registry.test.ts
    ├── render.test.tsx
    ├── security.test.tsx
    └── determinism.test.tsx
```

11. Important Observations from Task 4

11.1 Contract terminology was preserved

The board contract uses specVersion "1.0", page "landing", contextId and blocks containing component/props. It has no root/children tree, generated_for metadata, onboarding segment fields or refusal variant. The adapted prompt uses the current contract rather than tutorial examples.

11.2 Static security scanning requires careful test design

The original guide reports a scan matching a forbidden keyword inside a test string. That is tutorial history, not a board-project test result. For this project, distinguish executable code from test data and documentation, and combine static checks with behavioural probes. Do not weaken a control solely to obtain a green test.

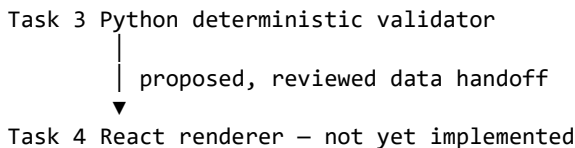
11.3 Package housekeeping

If later implementation installs UI packages, preserve the lockfile and ignore node_modules and build output. This document adds no UI package installation. Current Git object errors prevent reliable checkpoint claims until repository health is restored through a separately agreed action.

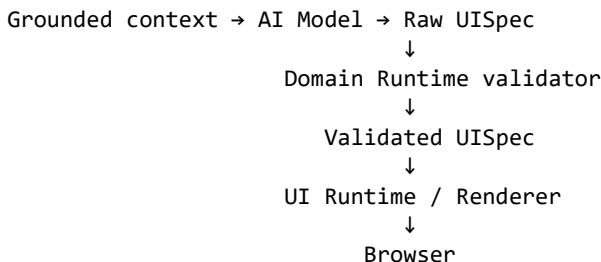
12. The Important Clarification Before Checkpoint

Task 3 exists locally as a Python validator. Task 4 is still an implementation brief. A future renderer increment must not be described as live Python-to-browser integration.

CURRENT:



FUTURE INTEGRATED FLOW:



The current validator returns a result, not a browser-trusted payload. Agree fixture provenance and the ownership of the validated object before implementation. A future API must bind payload, context and schema version; its transport and orchestration remain outside Task 4.

The two schemas differ in bounds, the selected schema has a conflicting value-length description, and architecture's omission wording differs from strict renderer rejection. Hero.primaryAction has no accompanying label prop. These are review items, not permission to invent fields or behaviour. See the Task 3 report and schema notes.

13. What Task 4 Does NOT Build

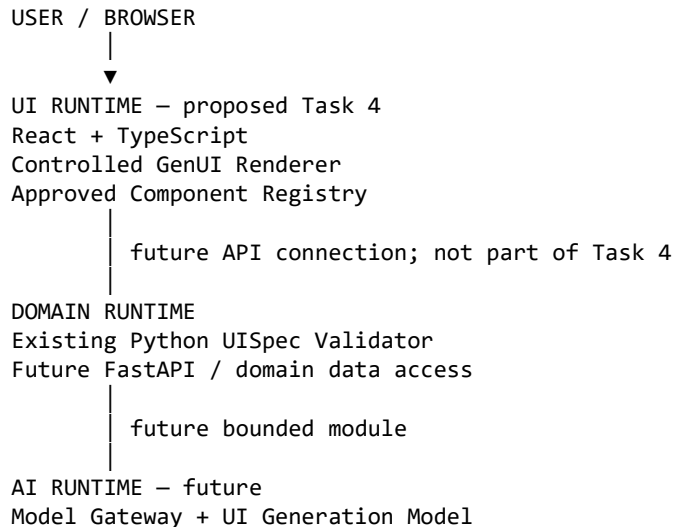
- FastAPI application runtime or validator-to-React API integration.
- Model gateway, live AI integration, retry/repair or fallback orchestration.
- Authentication or production infrastructure.

- Physical camera integration or edge image processing.
- Real-time PLC/MES signal handling or automated defect classification.
- Human review, override, disposition or board-quality approval workflows.
- Customer onboarding, document processing or risk scoring.
- Model evaluations, feedback loops or the full inspection product.

The BRD discusses wider inspection needs and open acceptance targets. Those requirements inform the business context; they do not expand this renderer increment. Quality and Manufacturing retain ownership of taxonomy, thresholds, ground truth and operating decisions.

14. Enterprise Architecture View After Task 4

The diagram shows the intended position of a completed renderer. It is not a statement that all runtime services exist.



The broader architecture documents include edge/enterprise deployment, evidence workflows and adaptive UI proposals. They are reference material for later increments, not an instruction to implement those systems here.

15. The Key Engineering Lesson

Generative UI does not require a model to generate executable UI code.

Model proposes a structured description



Validator checks compliance against supplied context



Registry selects a permitted implementation



Developer-authored React components render it

This separates presentation proposals from executable capabilities. Neither rendering nor a VALID result establishes that a board meets quality requirements. Semantic model evaluations remain distinct from deterministic software tests.

16. Task 4 Checkpoint Readiness

Status: guide prepared; implementation not started. The following are future review criteria, not completed claims:

- Schema authority, failure policy and approved-input handoff are agreed.
- Hero action-label behaviour is defined without changing props silently.
- Registry contains exactly the nine approved components.
- Renderer accepts verified input and protects its own registry/DOM boundary.
- All five approved action IDs are covered without invented destinations.
- Actual renderer, TypeScript, lint and Python regression checks pass.
- No live API, model integration or board-decision capability is claimed.
- Local diff and untracked files are reviewed; Git errors are resolved or reported.
- Task 4 is reviewed before any separately authorised commit or next task.

17. What Comes Next

Review this adapted brief and the unresolved readiness decisions first. Only then authorise the smallest controlled renderer implementation. Do not implement Task 4 merely because this document contains its prompt, and do not start Task 5 or an API/model integration automatically.

Source and adaptation notes

Format source: Customer_Onboarding_Task_4_Beginner_Guide_Enterprise_Standard.docx.pdf, 14 pages. The original editable DOCX was not supplied. This adaptation reconstructs its US Letter layout, heading hierarchy, 17 sections, bullet lists, monospaced diagrams and multi-page prompt. Exact original Word styles and pagination cannot be guaranteed from PDF alone.

Business and current authority: Documents/Business Requirements Document.docx; docs/business-problem.md; docs/architecture.md; CLAUDE.md; specs/spec.md; specs/model-contracts.md; both schema files and schema documentation; services/domain/README.md; specs/TASK-3-IMPLEMENTATION-REPORT.md; tests/README.md and evals/README.md.

Supporting Documents/ references reviewed: initial engineering discussion; first-implementation and Task 1 guides; Task 2 prompt/findings/clarifications; adapted Task 3 guide; gap analysis; user journeys; PRODUCT_BACKLOG; end-to-end workflow; solution/probabilistic architecture descriptions; proposed project structure and original onboarding guides. Earlier review-panel/ref-based examples and stale partial-schema statements are historical or alternative proposals, not the current v1 landing-page contract. Architecture images accompany these descriptions rather than supplying new contract rules.

One-Sentence Summary

Task 4 would turn a validated board-project UISpec into controlled React presentation through an explicit registry, without letting the model generate executable code or make board-quality decisions.